

AtCoder Beginner Contest 466

菜狗の赛后讲解

GzsJAY

2026/7/11

目录

1 A

2 B

3 C

4 D

5 E

6 ?

Problem A

给你一个数组判断其中有没有负数
遍历数组即可

```
void solve() {  
    cin >> n;  
    rep(i, 1, n) {  
        int x;  
        cin >> x;  
        if (x ≥ 0) {  
            cout << "No" << endl;  
            return ;  
        }  
    }  
    cout << "Yes" << endl;
```

目录

1 A

2 B

3 C

4 D

5 E

6 ?

Problem B

一共有 n 个球，给出每个球的颜色和大小，让你给出每种颜色的球的最大大小

- 以颜色映射大小使用桶数组，在输入颜色时将桶数组为该输入颜色的大小与输入的大小去 $\max$ 即

$$to[c[i]] = \max(to[c[i]], s[i])$$

Problem B

一共有 n 个球，给出每个球的颜色和大小，让你给出每种颜色的球的最大大小

- 以颜色映射大小使用桶数组，在输入颜色时将桶数组为该输入颜色的大小与输入的大小去 $\max$ 即

$$to[c[i]] = \max(to[c[i]], s[i])$$

```
void solve() {  
    memset(to, -1, sizeof(to));  
    cin >> n >> m;  
    for (int i = 1; i ≤ n; i++) {  
        cin >> c[i] >> s[i];  
        to[c[i]] = max(to[c[i]], s[i]);  
    }  
    for (int i = 1; i ≤ m; i++) {  
        cout << to[i] << ' ';  
    }  
}
```

目录

1 A

2 B

3 C

4 D

5 E

6 ?

C

交互题，给出 n 个点在数轴上，你可以提问 i 与 j 两个点之间的距离是否小于等于 1，并在 $2n$ 次询问中得到该数轴上距离小于等于 1 的个数

- 点从左到右排列，距离随 j 单调递增

C

交互题，给出 n 个点在数轴上，你可以提问 i 与 j 两个点之间的距离是否小于等于 1，并在 $2n$ 次询问中得到该数轴上距离小于等于 1 的个数

- 点从左到右排列，距离随 j 单调递增
- 用双指针维护窗口 $[l, r]$

C

交互题，给出 n 个点在数轴上，你可以提问 i 与 j 两个点之间的距离是否小于等于 1，并在 $2n$ 次询问中得到该数轴上距离小于等于 1 的个数

- 点从左到右排列，距离随 j 单调递增
- 用双指针维护窗口 $[l, r]$
- r 只增不减，总查询 $O(N)$

C

交互题，给出 n 个点在数轴上，你可以提问 i 与 j 两个点之间的距离是否小于等于 1，并在 $2n$ 次询问中得到该数轴上距离小于等于 1 的个数

- 点从左到右排列，距离随 j 单调递增
- 用双指针维护窗口 $[l, r]$
- r 只增不减，总查询 $O(N)$
- 答案 $= \sum_{i=1}^N (R_i - i)$

- 如何求 R_i 呢

- 如何求 R_i 呢
- 以 i 作为 l 与 r 构成区间，并循环查询区间长度是否小于等于 1，在查询失败前一个节点即为 r 的最大值

- 如何求 R_i 呢
- 以 i 作为 l 与 r 构成区间，并循环查询区间长度是否小于等于 1，在查询失败前一个节点即为 r 的最大值
- 即代码为

```
9 void solve() {
0   cin >> n;
1   ll ans = 0;
2   int r = 1;
3   for (int l = 1; l ≤ n; l++) {
4       if (r < l) r = l;
5       while (r < n && ck(l, r + 1)) {
6           r++;
7       }
8       ans += r - l;
9   }
0   cout << "! " << ans << endl;
1 }
```

目录

1 A

2 B

3 C

4 D

5 E

6 ?

有一个 $N * N$ 的 Grid，进行 M 次操作，每一次操作分别是删除第 r_i 行所有旗子与删除第 c_i 行所有旗子，随后在 (r_i, c_i) 放置旗子

- 我们可以转换成一个数学问题，维护两个分别为行和列的集合，我们命名他们为 S_r 与 S_c
 $S_r = \{r \mid \text{第 } r \text{ 行在 } i \text{ 之后被删除过}\}$
 $S_c = \{c \mid \text{第 } c \text{ 列在 } i \text{ 之后被删除过}\}$

有一个 $N * N$ 的 Grid，进行 M 次操作，每一次操作分别是删除第 r_i 行所有旗子与删除第 c_i 行所有旗子，随后在 (r_i, c_i) 放置旗子

- 我们可以有一个很方便的维护，若 $R_i \notin S_r$ 且 $C_i \notin S_c$ ，则棋子保留， $ans++$

有一个 $N * N$ 的 Grid，进行 M 次操作，每一次操作分别是删除第 r_i 行所有旗子与删除第 c_i 行所有旗子，随后在 (r_i, c_i) 放置旗子

- 我们可以有一个很方便的维护，若 $R_i \notin S_r$ 且 $C_i \notin S_c$ ，则棋子保留， $ans++$
- tips: 需要倒序操作，因为如果从前往后维护集合无法预判接下来是否有删除操作关联本次操作

目录

1 A

2 B

3 C

4 D

5 E

6 ?

有 N 幅卡片，正背面分别写着数字 A 与 B ，你有 k 次机会翻转 $[l, r]$ 区间的卡片，求最终朝上的数字总和的最大值。

- 考虑 dp 做法求翻转可得收益

- 5 后面再加上初始正面朝上的总和即可获得题目要求的答案

有 N 幅卡片，正背面分别写着数字 A 与 B ，你有 k 次机会翻转 $[l, r]$ 区间的卡片，求最终朝上的数字总和的最大值。

- 考虑 dp 做法求翻转可得收益
- 我们设 $dp[i][j]$ 为当前处理到第 i 个位置，翻转了 j 次有最大收益

- 5 后面再加上初始正面朝上的总和即可获得题目要求的答案

有 N 幅卡片，正背面分别写着数字 A 与 B ，你有 k 次机会翻转 $[l, r]$ 区间的卡片，求最终朝上的数字总和的最大值。

- 考虑 dp 做法求翻转可得收益
- 枚举 $i \in [1, n]$ ，每次枚举有两种决策方案
 - 1: 不以 i 结尾翻转任何区间

$$dp[i][j] = dp[i-1][j]$$

- 2: 以 i 结尾翻转一个区间 $[l, i]$

$$dp[i][j] = \max_{1 \leq l \leq i} \left(dp[l-1][j-1] + \sum_{x=l}^i (B_x - A_x) \right)$$

- 5 后面再加上初始正面朝上的总和即可获得题目要求的答案

有 N 幅卡片，正背面分别写着数字 A 与 B ，你有 k 次机会翻转 $[l, r]$ 区间的卡片，求最终朝上的数字总和的最大值。

- 考虑 dp 做法求翻转可得收益

- 用前缀和与差分数组优化，即可将 $\sum_{x=l}^i (B_x - A_x)$ 操作复杂度优化到 $O(1)$
- 5 后面再加上初始正面朝上的总和即可获得题目要求的答案

```
void solve() {
    memset(dp, -inf, sizeof(dp));
    cin >> n >> k;
    for (int i = 1; i ≤ n; i++) {
        cin >> a[i] >> b[i];
        prea[i] = prea[i - 1] + a[i];
        preb[i] = preb[i - 1] + b[i];
        val[i] = preb[i] - prea[i];
    }
    for (int i = 0; i ≤ n; i++) {
        dp[i][0] = 0;
    }
    for (int j = 1; j ≤ k; j++) {
        int tmp = -inf;
        for (int i = 1; i ≤ n; i++) {
            tmp = max(tmp, dp[i - 1][j - 1] - val[i - 1]);
            dp[i][j] = max(dp[i - 1][j], tmp + val[i]);
        }
    }

    int ans = 0;
    for (int i = 0; i ≤ k; i++) {
        ans = max(ans, dp[n][i]);
    }
}
```


目录

1 A

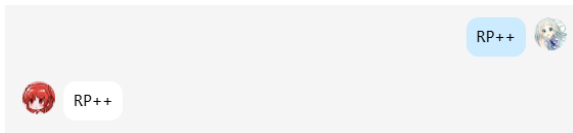
2 B

3 C

4 D

5 E

6 ?



图：